PRESENTATION DOCUMENT ÀIRNEIS

Table of Contents

Presentation of the team 4

Log Follow-up 4

A – Information to access the code 5

A.1 – Github 5

A.2 – Figma 5

A.3 – Jira 5

B – Introduction 5

B.1 – The context of the project 5

B.2 – The progress of the project 6

C – Choice of technologies 9

C.1 - Front-End 9

Context 9

Web - Solution 9

Web - Routing 10

Web - Notes: 10

Mobile - Solution 10

C.2 – Back-End 11

Context 11

Solution 11

Web Framework 12

D – Choice of architecture and security 12

D.1 – Database 12

SQL or PostgreSQL? Which one should we choose? 12

Solution 13

D.3 – Docker 14

What does it allow us to do? 14

Benefits 14
D.2 – Securing our Web Server 14
D.3 – Securing our CI/CD 14
D.4 – Summary Diagrams 15
E – Quick notes 16
F – Recap 16
G – The difficulties encountered 17
H – Conclusion 17

Presentation of the team

NAME	USERNAME GITHUB	EMAIL	ROLE
Magali DESLOUS-PAOL	Magali-D	magali.paoli@tutanota.com	SRE
Kylian BARTHELEMY	KyDev25	kylian.barthelemy2003@gmail.com	Product Owner
Cédric GAUTIER	cedricgautier	gautiercedric07@gmail.com	Security Engineer
Barnabé PILLIAUDIN	BarnabePILLIAUDIN	barnabe.pilliaudin@gmail.com	Developer Full-Stack
Alix BERTHELOT	hallycsse	alixberthelot03@gmail.com	Developer Full-Stack

Log Follow-up

WHO	WHAT	WHEN
Kylian BARTHELEMY	Start of drafting of the document	05/06/2024
Kylian BARTHELEMY	Completing the document	10/06/2024

A – Information to access the code

A.1 – Github

To access Github, follow the link below:
https://github.com/KrispyTech/airneis
Github contains all the architectures for the project. You can read the README.md in the '*Getting started*' chapter to find out how to start the project.

A.2 – Figma

To access Figma, follow the link below:
https://www.figma.com/file/tqpdNMq2MpwY135z4RKXi3/AIRNEIS?type=design&node-id=2%3A2&mode=design&t=6YN6TcDGJsWQlcyF-1
Figma contains all the models, components and pages of our web and mobile application.

A.3 – Jira

To access Jira, follow the link below:
https://kylianbarthelemy.atlassian.net/jira/software/projects/AIRNEIS/boards/2
Jira contains all the information on user stories and epics that we have used to organize our work.

B – Introduction

B.1 – The context of the project

ÀIRNEIS asked us, in a context where digital technology is constantly evolving, to create a web and mobile e-commerce application for the resale of Scottish furniture.

In this context, at the start of the project, we decided together to use agile methodology with the Scrum and Kanban frameworks. Thanks to the agile methodology and its frameworks, we were able to monitor our progress in real time and organize our work into several one-month sprints.

Then we used **Github** to have a clean architecture and manage the processes to put the code in the dev or prod environment. Thanks to Github and its processes, we were able to develop in a good environment and manage our code easily.

Once we had a good architecture to develop, we created a **Jira** to organize our user stories. Thanks to Jira, we were able to use the agile methodology.

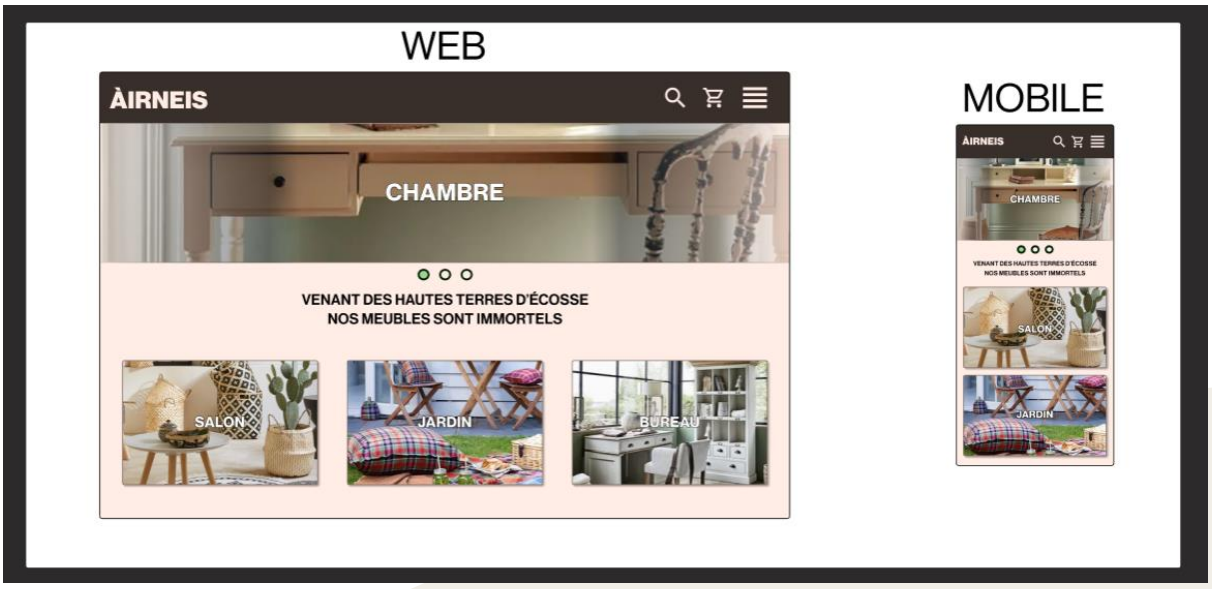
Finally, we used **Figma** to create our components and pages. Thanks to Figma, we were able to get an overview before development and save time.

It was in this context and organization that we developed the web and mobile application requested by ÀIRNEIS.

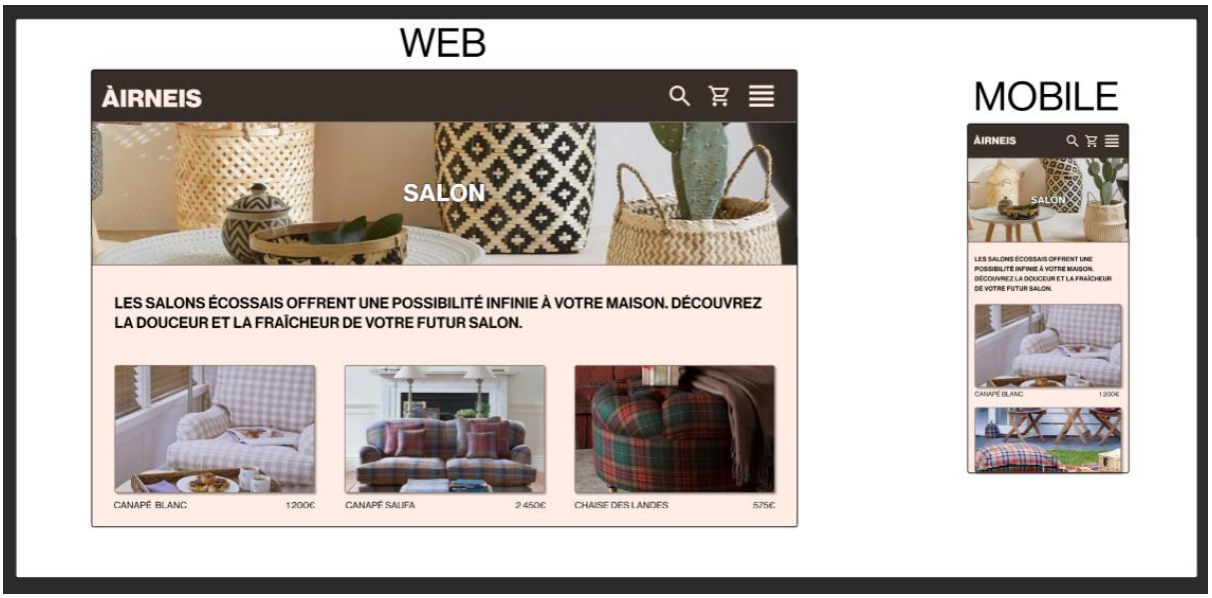
B.2 – The progress of the project

Now, our main objective is to provide ÀIRNEIS with a usable application with all the necessary functionality so that users can take full advantage of the web or mobile application. That is why, at the end of the six months, our aim is to be able to allow users to log in, browse the web and mobile application to look at the categories and access all the information about a product. Finally, we will try to ensure that the user can access the cart to order a product in total security. This option will depend on our progress.

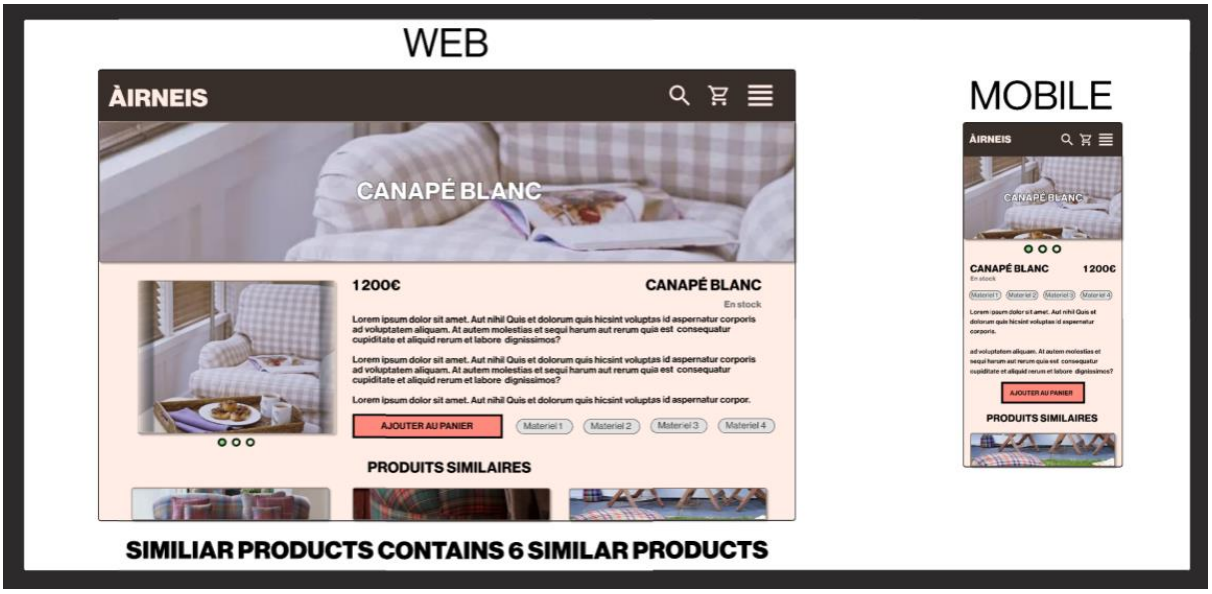
For the moment, here is the progress of our project, which will evolve and be completed once the objective has been reached:



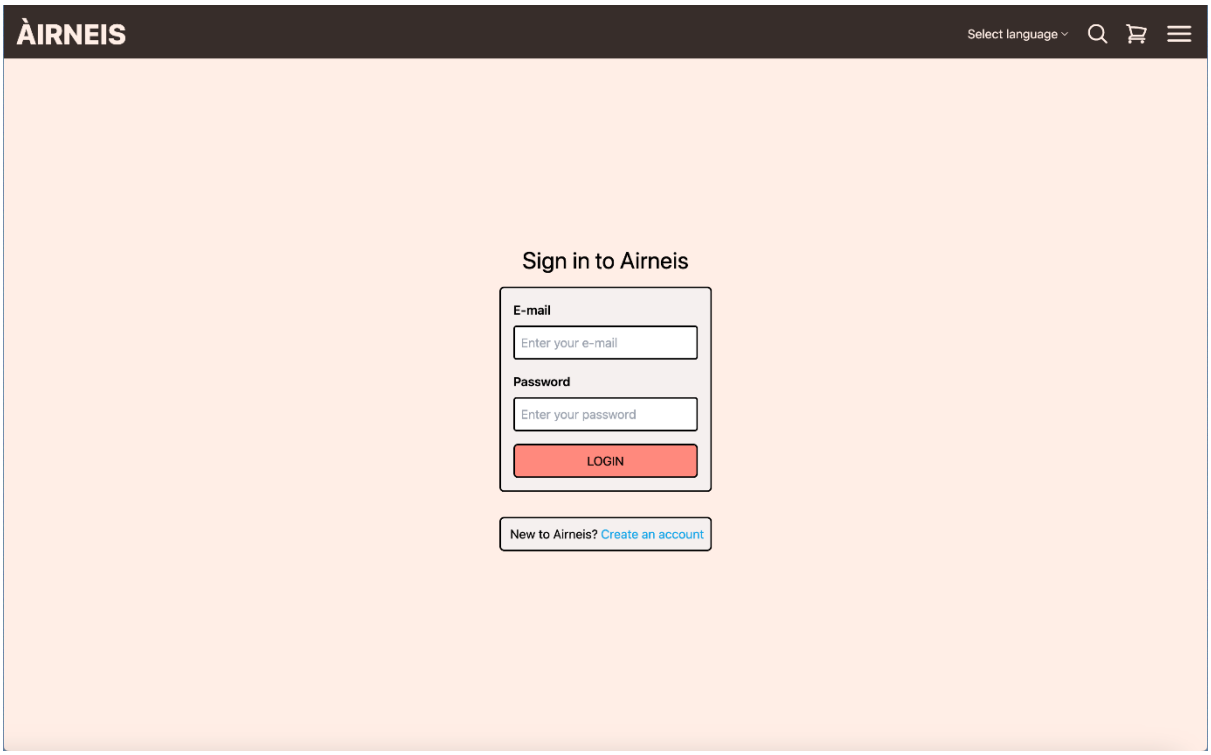
This is the main page for viewing the categories



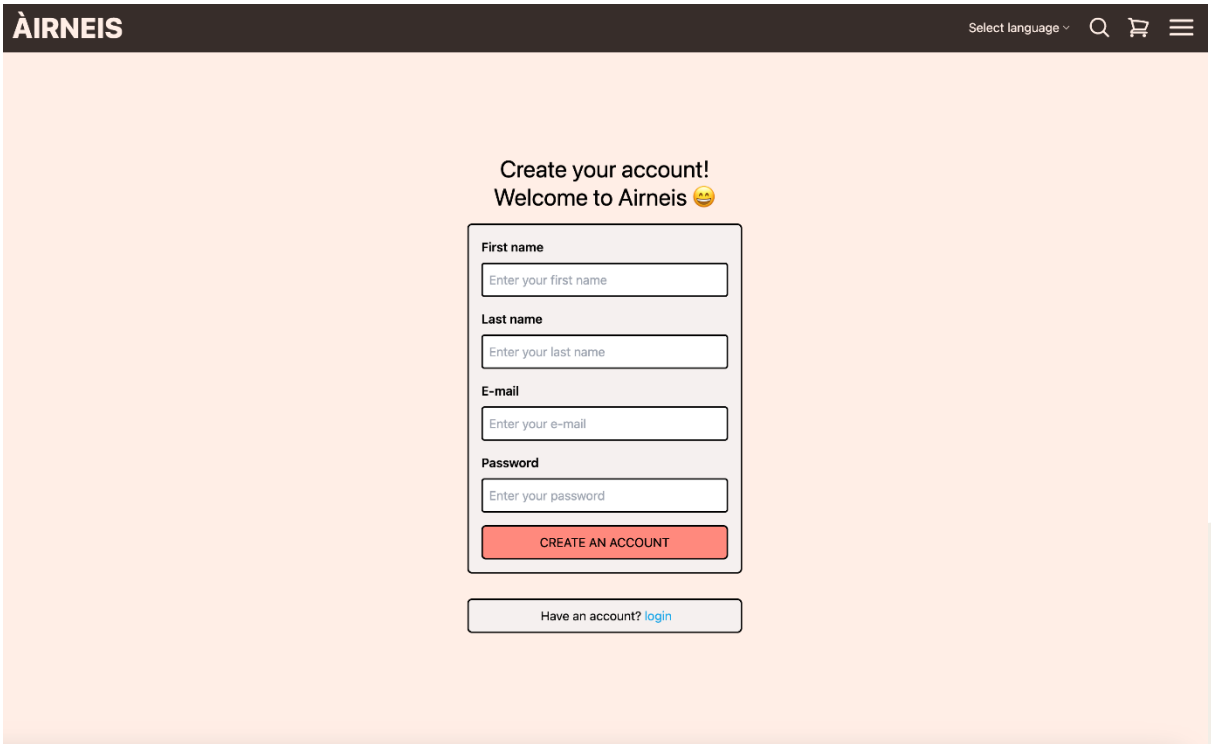
This is the category page to view the products in a category



This is the product page to view all information about a product

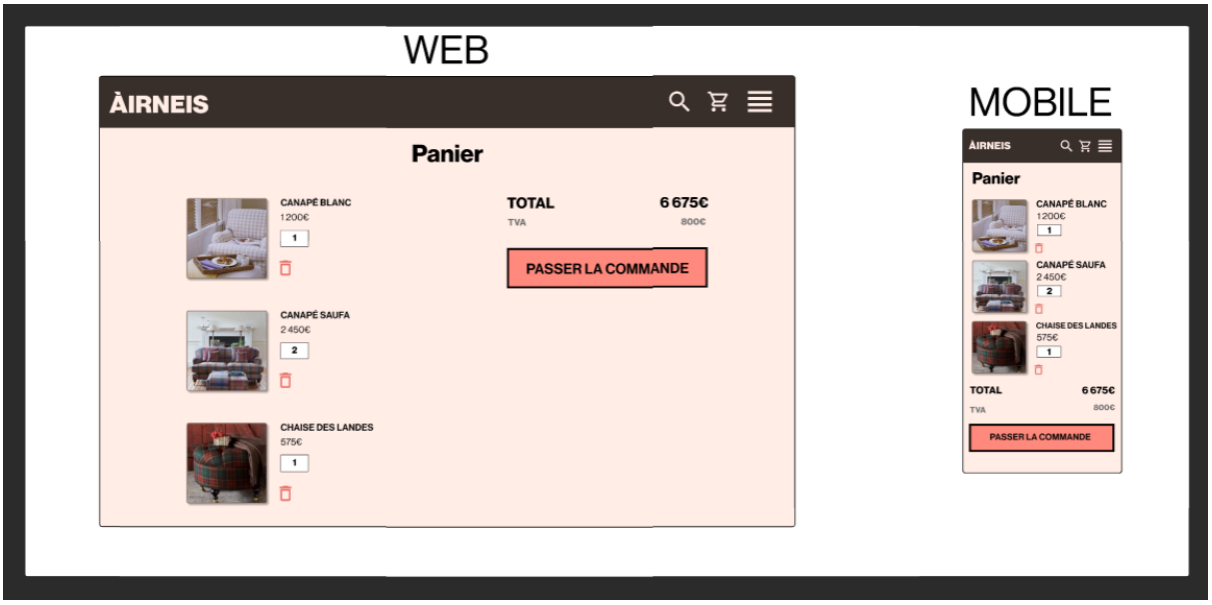


This is the login page to create an account



This is the register page to access the full web or mobile application

Here is the work we are currently doing above. For a future preview of the cart so that users can manage their cart, here is a mock-up created on Figma, bearing in mind that it has not yet been developed:



This is the future cart page to manage and buy several products

To sum up, we are currently **at 60%** of our target.

C – Choice of technologies

C.1 - Front-End

Context

This part of the stack controls the data that is displayed on our app, we will need a library for building user interfaces for our users.

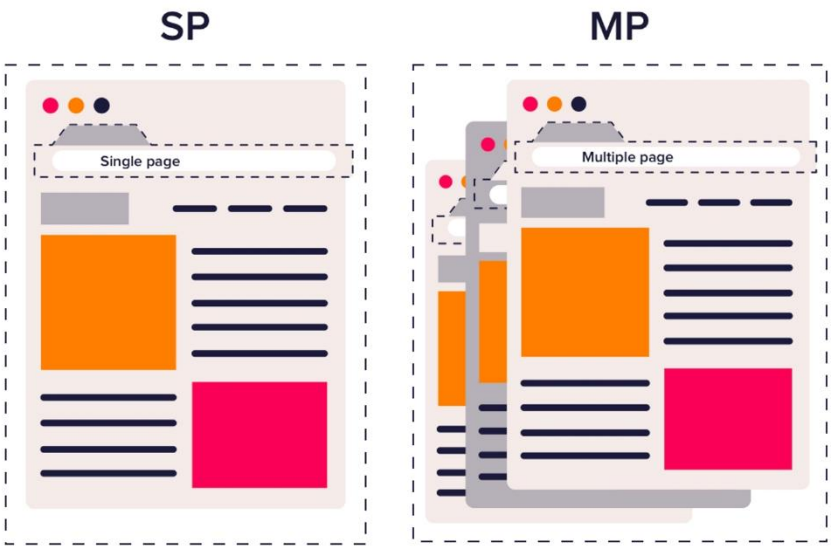
Web - Solution

We will be using **NextJS** for these main reasons:

- We can create our custom components, which we will need when creating in our app as we are asked to have our own Design System.
- NextJS uses a Single Page Application to speed up processing.

The image above shows the difference between a SPA and an MPA. You can see that the MPA reloads the entire page for each tab. In our case, React JS with NextJS uses the SPA to avoid reloading the page between each tab with the use of the Virtual DOM which allows to

change just the difference between the current state and the previous state. The image below represents how the virtual dom works.



Single Page Application and Multi Page Application differences

Web - Routing

We'll be using NextJS' App Router instead of using the Pages Router to adopt the new routing system of NextJS but it's good to know that it can work both alongside each other but the App Router takes priority over the Pages Router in the case there is a duplicate page:

- Possibility to have dynamic routes in the case we need to have a page for a specific type of object `*eg: [nameOfComponent].js*`

Web - Notes:

As NextJs has a system of nested layouts we can have a component on each children page by putting it into layout.js. The component will be displayed in the children without having to mention it in the children page.js.

NextJs can handle both server side and client-side rendering on the same app.

Mobile - Solution

We will be using **NativeWind**. NativeWind uses Tailwind CSS as scripting language to create a universal style system for React Native. NativeWind components can be shared between platforms and will output their styles as CSS StyleSheet on web and StyleSheet.

C.2 – Back-End

Context

This part of the stack controls the data traffic of our app, meaning how data is fetched, pushed, modified, or deleted on our database. We will need a Web Framework to build an API, a database and possibly an ORM to have a bridge from native SQL queries to our selected language.

In our case, we need backend that is blazing fast with the best support on large amounts of traffic. We are junior developers, so let us take an approach of simplicity and minimalism for the structure of our code and have an expressive way of coding for easy reading.

Solution

Creating an app can be a big challenge I would like for us to have a philosophy on exactly what is happening in our code.

To make it as explicit as possible, knowing what type(s) of variable(s) enters a function and what it type(s) it returns to be able to have the best comprehension of how everything works.

"Go was designed at Google in 2007 to improve programming productivity in an era of multicore, networked machines and large codebases. The designers wanted to address criticism of other languages in use at Google, but keep their useful characteristics"

Go is easily picked up as the language is similar to Python and C, to easily define the language it's like Python and C made a baby having in mind performance (performance of C) without the cost of code complexity (easy to read like Python).

The founders took the best of both worlds, having in mind a simple, efficient statically typed language with reduced complexity.

Words from a JS developer who switched to Golang - Steve Hook:

"Go is transparent, it is plain and simple, sometimes stupid, which makes no place for gotchas and magical code, which is a norm in dynamic languages such as JavaScript. With this mindset, and having some experience in this language"

Web Framework

Fiber is an Express inspired web framework built on top of Fast HTTP, the fastest HTTP engine for Go. Designed to ease things up for fast development with zero memory allocation and performance in mind:

- Fastest Golang Web Framework.
- Fiber is using HTTP1.1 Natively but support for HTTP2 is possible.
- Fast HTTP has no plans to support HTTP2 in the future.

But we may be able to do something with Cloudflare as it supports.
With our team having some experience with the Express Framework, this will be an easy transition on using Fiber. Everything is practically the same, but we will be using the Go language. Additional information about Fiber:

- Package Scores
- Fiber Tutorial
- Express beaten by Fiber, article with benchmarks
- Fiber Welcome Page
- Fiber Documentation

D – Choice of architecture and security

D.1 – Database

We will use a relational database in order to be able to relate one object to establish links or relationships–between information by joining tables, which makes it easy to understand and gain insights about the relationship between various data points.

SQL or PostgreSQL? Which one should we choose?

Microsoft SQL is a product owned by Microsoft, SQL Server is available for use through a commercial core-based license as a Standard or Enterprise edition with prices ranging from $3,586 to $13,748. There are also two free versions, a full-featured edition for developers available for non-production workloads and a free Express edition with limited features and database sizes.
PostgreSQL is open source, released under the PostgreSQL License. This means there is no fee for using this product for any purpose, including commercial use. According to the PostgreSQL Global Development Group, PostgreSQL will remain free and open source in perpetuity and there are no plans to change the license or release the product under a different license.

Solution

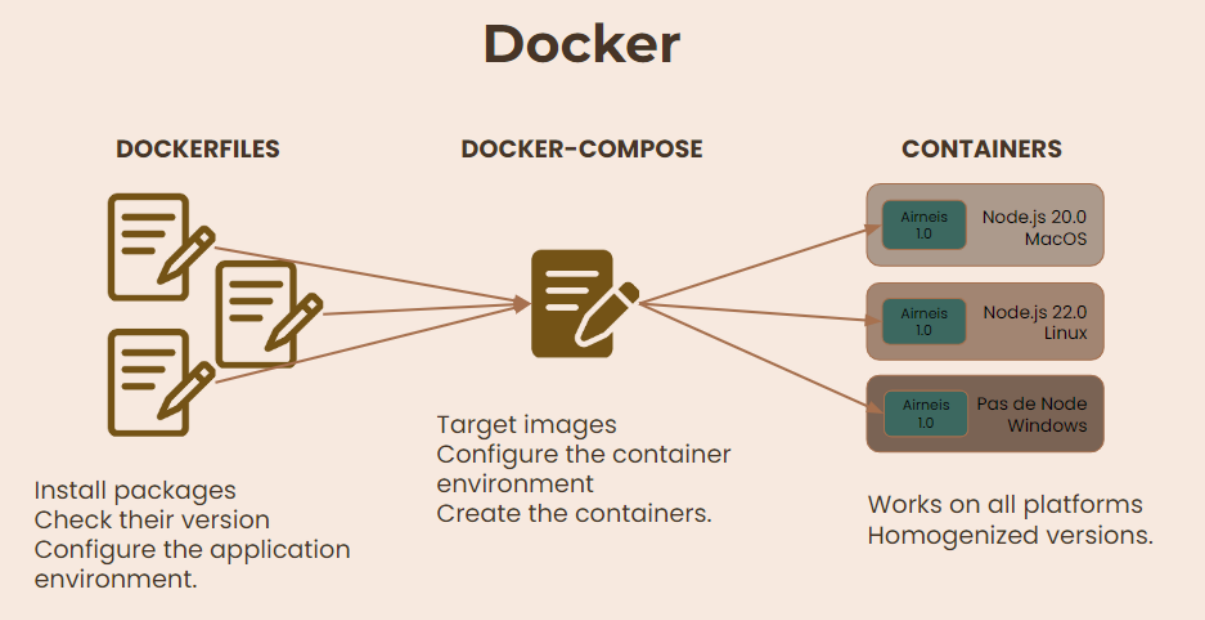
With the intent or our product being commercialized, having PostgreSQL will reduce our cost fees as there is no fee for use in commercial software products.

Here is our database schema with PostgreSQL:



D.3 – Docker

What does it allow us to do?



Benefits

There are three main advantages:

- **Portability:** Compatible with all environments, no additional configuration.
- **Speed:** Dev or Prod deployment in seconds.
- **Lightness:** Back-end Docker image without multistaging (1.23GB) and with multisaging (15.8MB).

Docker allows us to optimize our environment for developers.

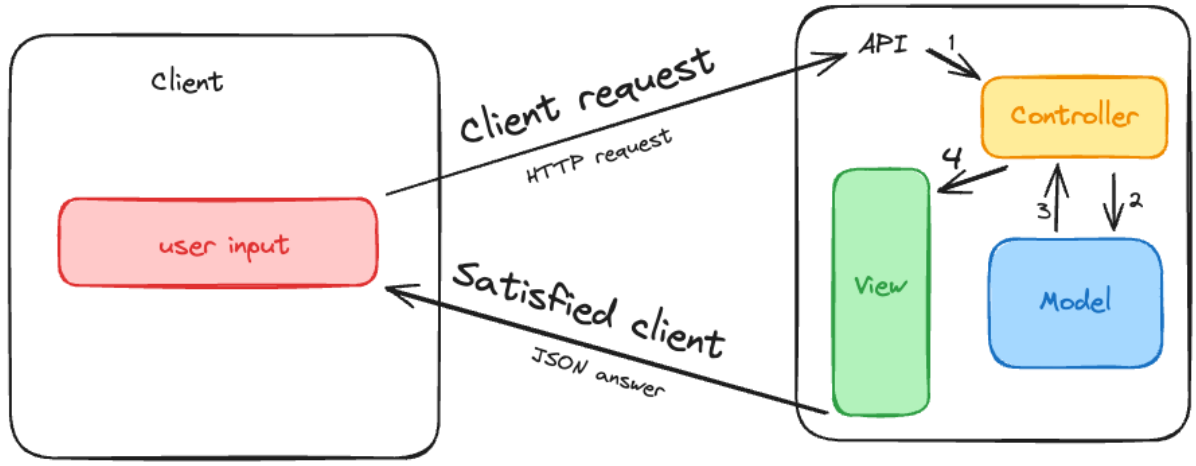
D.2 – Securing our Web Server

Add **Cloudflare** in front of web server to block for unwanted traffic and limit any DDOS Attack.

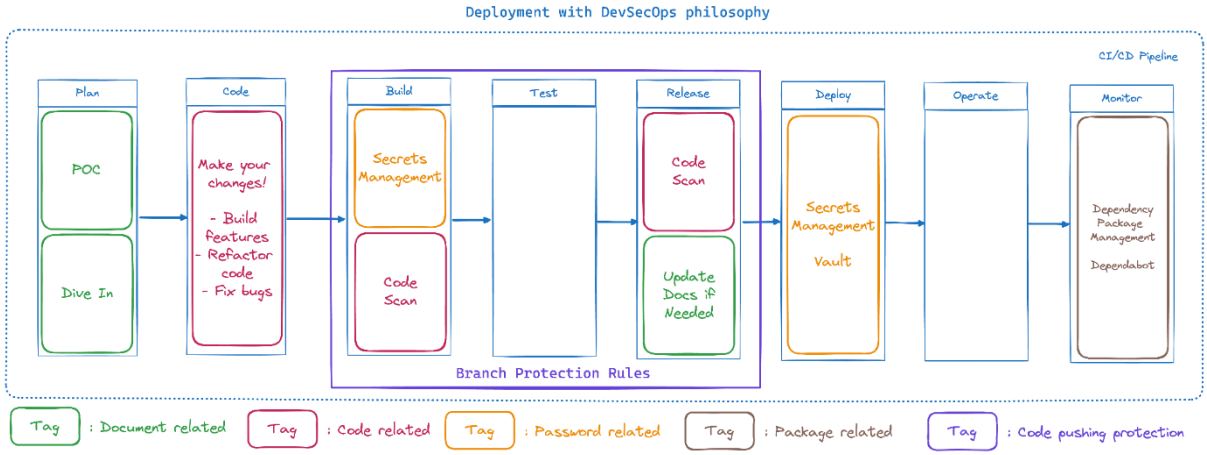
D.3 – Securing our CI/CD

Add a **CI** to be integrated before being able to merge requests - Add lingting check : Custom JS Lint and golangci-lint - Add testing phase (optional) - Gingko & Gomega go test ../ - Add injection on required environment variables from CI Variables - Add secret detection phase.

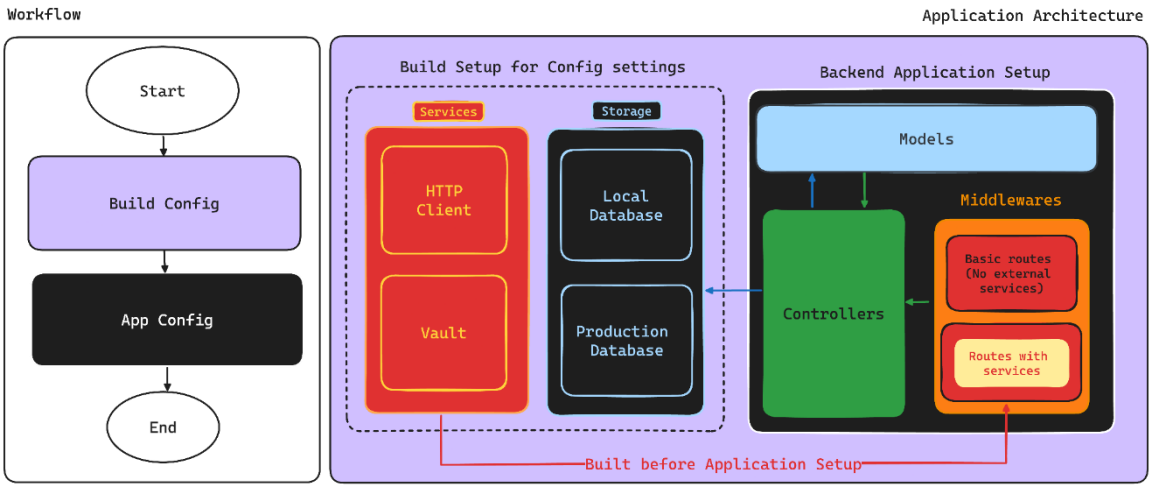
D.4 – Summary Diagrams



MVC architecture



Deployment with DevSecOps philosophy



Operation of our maintainable and secure application

E – Quick notes

- Think of what the cost is, what is the environment and what you are really going to be end up doing. Better to know your environment and product before pushing something you will regret.
- You need a migration service for your PostgreSQL database, do not use vanilla SQL.
- Creating a CI with an amazing pipeline has bonus points.
- Use of containers to be able to deploy and dev comfortably.
- You are going to need to Use NEON, a serverless function for PostgreSQL.
- We could use Storybook to build UI components and pages in isolation. (It will be a bonus we are not sure to implement it).
- We could get secrets securely if needed on docker containers by fetching them from Hashicorp Vault.

F – Recap

Frontend: Javascript - NextJS

Backend: Go - Fiber

Mobile: NativeWind (Tailwind with React Native)

Database: PostgreSQL

VCS: Github

CI/CD: Github Actions

Storage: TBD

Secret Management: Hashicorp Vault

Web Proxy: Cloudflare (probably if needed)
Deployment: TBD

G – The difficulties encountered

During the first few sprints, there was a lack of responsiveness within the group in terms of messages and communication, so it was always the same people who were involved in the project and there was a feeling of loneliness and of a task waiting to be answered. After discussion, we all tried to communicate better so that we could move forward together on the project.

In the following sprints we encountered more of a problem with reviews on Github. A lot of pull requests had been open for too long, which was blocking the progress of the project. To solve this problem, we had to remove the principle of two reviewers and put just one. We have also made efforts to be more present for reviews and to be more precise about the actions to be taken for modifications.

We also had poor organization in terms of the distribution of tasks. Tasks were too dependent on other tasks, which wasted time on the progress of the project. To solve this problem, tasks were more independent and better distributed among the teams.

We also encountered difficulties in terms of development despite a complete and motivated team. For example, Atlas and Ent were not configured in time, leading to delays in database and backend development. Or, Hashicorp changed its Vault authentication system. A major overhaul had to be done on the Vault, which cost the project a lot of time. Finally, the session also put us in difficulties which made us waste time on setting up the next objectives. We therefore organized ourselves to catch up and be able to continue our path.

Despite all the difficulties encountered, we managed the situations and remained united to help each other and move forward together.

H – Conclusion

All the fruit of our work is due to several factors:

- Our technical stack which allowed us to build a stable development environment with many advantages and time savings.

- Our organization which allowed us to create a secure and peaceful architecture and development environment.
- Our architecture which allowed us to create clean and maintainable development.

All this was possible thanks to a team which, despite the difficulties and drops in motivation, remained united, with a transparent and attentive organization.

Each member was present and did the best they could and that is the most important thing to remember.